

A Predictive Digital Twin Framework for Real-Time Health Monitoring of Electric Battery Systems

Kaushalya Thopate
Assistant Professor,
Computer Engineering,
Vishwakarma Institute of Technology
Pune, India
kaushalya.thopate@vit.edu

Parag Panzade
Computer Engineering,
Vishwakarma Institute of Technology
Pune, India
parag.panzade24@vit.edu

Krishna Paryani
Computer Engineering,
Vishwakarma Institute of Technology
Pune, India
krishna.paryani24@vit.edu

Parth Koul
Computer Engineering,
Vishwakarma Institute of Technology
Pune, India
parth.koul24@vit.edu

Vedant Pasarkar
Computer Engineering,
Vishwakarma Institute of Technology
Pune, India
vedant.pasarkar24@vit.edu

Abstract—Knowing when a battery is about to fail—before it actually does—is one of the more practical challenges in modern energy systems. This work describes a low-cost monitoring platform built around an ESP32 microcontroller that collects voltage, current, and temperature readings from 18650 lithium-ion cells and converts them, in real time, into an estimate of the battery’s State of Health (SOH). Rather than forwarding raw sensor values directly to a model, the system first expands six measurements into twenty derived parameters through a domain-informed feature pipeline. A Random Forest regressor, optimised by exhaustive grid search over 216 configurations, is then applied to these features. On 172 samples held back from training, the model achieves $R^2 = 0.9999$, RMSE of 0.0306%, and MAE of 0.0221%. The monitoring loop runs once per second, with each SOH inference completing in under one millisecond. Total hardware cost stays below fifty US dollars. Remaining Useful Life (RUL) estimates, produced alongside every SOH reading, were checked against ten cells discharged to end-of-life under controlled conditions; the mean absolute error across those cells was 2.96%.

Index Terms—battery health assessment, ensemble learning, embedded sensing, predictive analytics, energy storage monitoring, digital twin, IoT, random forest, state of health, remaining useful life

I. INTRODUCTION

Energy storage has quietly become load-bearing infrastructure for modern technology. Lithium-ion cells now sit inside everything from wearable electronics to grid-scale installations, and their dominance reflects genuine engineering merit—high energy density, low self-discharge, and falling production costs make them hard to displace [1], [10]. The difficulty is that every charge–discharge cycle nudges a cell slightly closer to failure. Capacity shrinks, internal resistance climbs, and the margin between a healthy pack and a hazardous one narrows in ways that are not always visible until something goes wrong [11], [12].

What actually happens inside an ageing cell is a convergence of several mechanisms. SEI layer thickening consumes

lithium that can no longer carry charge; lithium plating deposits metallic fragments that create internal short risks; active material particles crack and lose electrical contact. Individually each process is slow, but together they produce the measurable capacity fade that defines cell ageing [11], [15]. A battery that retains 80 % of its original capacity is generally accepted as still serviceable; below that threshold, performance becomes unpredictable and safety risks increase [1]. Tracking how much capacity remains relative to what the cell started with—what engineers call State of Health (SOH)—gives operators the information they need to act before problems occur.

Established SOH measurement techniques each carry practical drawbacks that limit field deployment. A full controlled-discharge test is accurate but interrupts service and requires a laboratory setup [1]. Electrochemical Impedance Spectroscopy resolves internal degradation signatures precisely but depends on impedance analysers that cost far more than the cells under test [9], [24]. Coulomb counting is computationally inexpensive yet accumulates integration drift that requires periodic correction [18]. Kalman-filter state estimators built on equivalent-circuit models lose accuracy as cell parameters shift with age, because the model and the real cell gradually diverge [12], [16]. None of these approaches is well suited to continuous, unattended monitoring of cells in the field.

Inexpensive microcontrollers with wireless radios have changed what is practically achievable in embedded sensing. An ESP32 costs under ten dollars, reads analog signals through a twelve-bit converter, and communicates over Wi-Fi without additional hardware. The harder problem is not acquisition but interpretation: turning a stream of voltage and current numbers into a reliable SOH estimate in real time [5]. That translation step is where machine learning is most useful. Data-driven models do not need an explicit electrochemical model of the cell; they learn the mapping between measurable signals and health state from observed data [10], [23]. Among the

available algorithm families, Random Forest regression stands out for tabular sensor inputs because it tolerates measurement noise well, needs no normalisation, and reports which features matter most—a property that matters when results must be explainable [3], [6].

Even so, most published systems stop short of a complete deployable solution. Benchmark results are frequently produced on laboratory cyclers datasets that are not available in field conditions [14], [21]. Feature pipelines that extract physically meaningful quantities from streaming measurements, rather than from full charge curves, receive comparatively little attention [5], [13]. And RUL estimates are seldom validated against batteries actually discharged to end-of-life; they are more often evaluated on held-out segments of the training data, which is a weaker test.

The work described here attempts to close those gaps. The platform combines an ESP32 sensor node with a twenty-feature engineering pipeline and a grid-search-optimised Random Forest. It updates every second, costs under fifty dollars to build, and generates each SOH prediction in under a millisecond on standard laptop hardware (measured on a commodity x86 CPU running scikit-learn’s compiled tree implementation). RUL estimates were checked against ten cells that were discharged all the way to the 80 % SOH retirement threshold.

A. Research Objectives

The work sets out to accomplish five things:

- 1) Build a hardware sensing layer that is genuinely affordable and suitable for continuous deployment in non-laboratory environments.
- 2) Expand raw electrical measurements into a richer feature set using domain knowledge about battery electrochemistry [5].
- 3) Select and optimise a regression model through rigorous cross-validated hyperparameter search rather than manual tuning [3].
- 4) Confirm that inference is fast enough for one-hertz monitoring on commodity hardware, and quantify the computational cost.
- 5) Validate RUL predictions against physical discharge experiments—not held-out training data—on ten independent cells [26].

B. Key Contributions

The concrete outputs of the work are:

- **A sub-fifty-dollar sensing node** built from the ESP32 platform with ACS712 current sensing and LM35 temperature monitoring, designed for continuous unattended operation [13].
- **A twenty-feature engineering pipeline** that derives internal resistance, rolling voltage statistics, thermal-electrical stress, and temporal rate-of-change from six raw inputs. The pipeline design follows the domain-knowledge principles documented by Lanubile et al. [5] and Feng et al. [9], adapted for real-time streaming data.

- **Systematic model optimisation** across 216 Random Forest configurations using five-fold cross-validation, following the evaluation protocol of Cervellieri [3].
- **A live monitoring dashboard** displaying seven channels with fault severity classification and RUL estimates updated each second.
- **Physical RUL validation** against ten cells discharged to the 80 % capacity threshold, achieving 2.96 % mean absolute error—comparable to the benchmark results reported by Nuhic et al. [26].
- **Transparent limitation reporting** covering target-leakage risk, feature approximation at inference, and training scope boundaries, consistent with the responsible ML reporting standards discussed by Khaleghi et al. [2].

The paper continues as follows. Section II surveys relevant prior work. Section III covers hardware and software architecture. Section IV describes the machine learning approach. Section V explains the RUL estimators. Section VI reports experimental results. Section VII discusses findings and limitations. Section VIII concludes.

II. LITERATURE REVIEW

A. The Limits of Conventional Diagnostic Methods

Researchers have been estimating battery health for decades, and the field has produced several well-understood measurement techniques. Each works well under the right conditions but encounters a practical barrier that rules it out for continuous low-cost monitoring.

Controlled-discharge testing establishes ground-truth capacity by draining a cell at a specified rate and measuring the delivered charge. The result is highly accurate, but the procedure takes hours, interrupts service, and requires a bench-top cycler [1]. EIS maps degradation at a finer level by applying a range of AC frequencies and analysing the impedance response; it can distinguish SEI growth from lithium plating. Jiang et al. [24] conducted a detailed comparison of EIS-derived features and found that impedance metrics carry information about cell ageing that time-domain voltage measurements cannot fully replicate. The limitation is cost: a capable impedance analyser may cost more than the battery pack it monitors, placing EIS firmly in the laboratory rather than the field. Coulomb counting—integrating current over time to track charge throughput—is computationally cheap and widely embedded, but integration drift accumulates with each cycle and requires periodic reset procedures [18]. Model-based state estimators using Kalman filters and equivalent-circuit models can produce real-time SOH estimates without full discharges, but their accuracy depends on model parameters that change as the cell ages [12], [16]. None of these methods combines accuracy, low cost, and continuous operation in a single package.

B. Data-Driven Methods: Themes and Trade-offs

Much of the recent literature replaces explicit electrochemical models with machine learning trained on historical cycling data. The core insight is that a model does not need to know

why capacity fades—it only needs to see enough examples of that fading to learn its signature in available signals. This shift has been enabled by open benchmark datasets. The NASA Prognostics Repository [14] and the Stanford cycle-life dataset [21] provide the kind of long-run cycling records needed to train and compare models, and a substantial fraction of the published work evaluates against them.

Within the data-driven literature, three families of models dominate. Neural networks—particularly LSTM variants—exploit temporal structure. Chemali et al. [19] were among the first to show that LSTM networks could estimate lithium-ion SoC accurately, and subsequent work extended the idea to SOH. Gong et al. [7] improved LSTM architecture design and reported strong performance on NASA benchmark cells. Peng et al. [8] took the further step of engineering multi-scale health features as inputs rather than feeding raw measurements, reducing error across several cycling protocols. Lu et al. [20] pushed the boundary further by demonstrating that deep learning can estimate SOH without running any additional degradation experiments, relying instead on charging curve shape. The pattern across these studies is consistent: LSTM models achieve high accuracy but demand substantial training data, GPU-class hardware for practical inference throughput, and produce predictions that are difficult to interpret or audit [2].

A second family uses convolutional neural networks to treat voltage–time discharge curves as one-dimensional signals. Li et al. [11] applied CNNs to extract spatial patterns from charge profiles and demonstrated that such patterns carry degradation information. Shen et al. [17] showed that combining convolutional feature extraction with ensemble learning and transfer learning can produce competitive capacity estimates, at the cost of significantly more complex training pipelines and larger data requirements.

The third family—ensemble methods including Random Forest and gradient boosting—prioritises robustness and interpretability. Cervellieri [3] benchmarked Random Forest regression specifically against SOH estimation tasks across several degradation scenarios and found it competitive with neural approaches on tabular data while using a fraction of the compute. Zhang et al. [6] went further by hybridising Random Forest and GRU: the forest extracts static health indicators, while the recurrent unit captures trajectory dynamics. Hu et al. [10] explored gradient boosting for battery lifetime prognostics and concluded that sequential ensemble learners can achieve low bias but need careful regularisation when data is limited. Roman et al. [23] demonstrated that a well-designed end-to-end ML pipeline—covering feature selection, model training, and evaluation protocol—can generalise across multiple cell chemistries, which is an important practical requirement.

The common thread in ensemble methods is a favourable trade-off for embedded deployment: they run in microseconds on a CPU, need no feature scaling, and let engineers inspect which inputs drive predictions [3]. Those properties matter when the goal is a field-deployable system rather than a benchmark competition.

C. What Features Actually Help

Whether the model is a neural network or a forest, the features fed to it shape what it can learn. Raw voltage and current carry useful information, but they do not represent degradation state as directly as derived quantities do.

Lanubile et al. [5] formalised this observation by testing a hierarchy of feature sets—ranging from raw measurements up to full differential voltage analysis and thermal metrics—and measuring the accuracy gain at each level. The conclusion was unambiguous: domain knowledge embedded in features substantially improves prediction accuracy, especially when transferring a model to a new cell type with little retraining data. Feng et al. [9] took a complementary approach, identifying which features extracted from partial charging curves carry the highest mutual information with capacity fade. Features with strong correlation to fade remained useful even across cells with different thermal histories.

In embedded settings, full charge curves are typically unavailable because the system is measuring a cell under real load rather than a controlled laboratory protocol [13]. This pushes feature design toward lightweight quantities computable from a sliding window of recent samples: rolling averages, rate-of-change estimates, and physics-inspired combinations like the thermal–electrical stress index. Shen et al. [17] showed that rolling statistical features computed from streaming telemetry retain meaningful degradation information even when the full waveform is not available. The present work applies all three layers of feature enrichment—rolling statistics, temporal derivatives, and physics-based interactions—in a real-time pipeline operating on one-second sensor windows.

D. IoT and Digital Twin Approaches

The availability of Wi-Fi-capable microcontrollers has opened a practical path for distributed battery monitoring. Wu et al. [13] reviewed published IoT architectures for EV battery management and identified a consistent design split: either all inference runs in the cloud (which introduces latency and connectivity dependency), or the device uses only simple threshold rules (which gives no SOH estimate). Neither option is ideal. Lipu et al. [4] surveyed ML-based SOH prediction methods and noted the importance of balancing model sophistication against the computational constraints of embedded platforms.

Digital twin frameworks address this by treating the software model as a continuously updated replica of the physical cell. Tian et al. [16] showed that a neural network battery model, updated from minimal charge data, can track SOH evolution closely over time. Wang et al. [12] used constant-voltage charging curves to feed a twin-like estimation system. The distinguishing characteristic of a digital twin—as opposed to a batch-mode analyser—is that it updates from live data and can be queried at any moment for the current state and a future forecast.

E. What Remains Unresolved

Across the literature, the combination of all of the following in one deployable system is rare: hardware that costs

under fifty dollars; a feature pipeline designed specifically for streaming (not offline) data; a model selected through principled hyperparameter search rather than default settings; and RUL validation against cells actually tested to end-of-life rather than against held-out training sequences [14], [26]. Each element appears in some papers but seldom all together. The present work attempts to assemble them into a single validated framework, with honest reporting of where the system’s assumptions may not hold [2].

III. SYSTEM ARCHITECTURE AND IMPLEMENTATION

A. Hardware Configuration

The sensor node is built around an ESP32 microcontroller—an Xtensa LX6 dual-core processor running at up to 240 MHz with a twelve-bit ADC and integrated Wi-Fi. Three modules connect to its analog inputs.

Voltage is measured through a resistive divider that accepts up to 25 V and attenuates by 5:1 before the ADC. The twelve-bit resolution translates to approximately 6 mV per step, which is four times finer than ten-bit platforms would provide.

Current is measured by an ACS712 Hall-effect sensor configured for the 5 A range. It outputs 2.5 V at zero current and shifts 185 mV per ampere in either direction, giving bidirectional measurement without shunt resistors.

Temperature is read from an LM35 linear semiconductor sensor that delivers 10 mV per degree Celsius across -55°C to $+150^{\circ}\text{C}$.

The cell under test is a standard 18650 cylindrical lithium-ion cell with a 3.7 V nominal voltage, 4.2 V full-charge voltage, and rated capacity of 2,200 mAh (2.2 Ah). This capacity figure is the denominator in all SOH and Coulomb-accounting calculations and is stored in `model_metadata.json`.

The choice of ESP32 over higher-cost platforms is deliberate. Wu et al. [13] identify this class of device as well suited to IoT battery monitoring because it combines adequate analog resolution with wireless connectivity at a component price below ten dollars. All signal conditioning uses passive components; no dedicated battery-management integrated circuit is required.

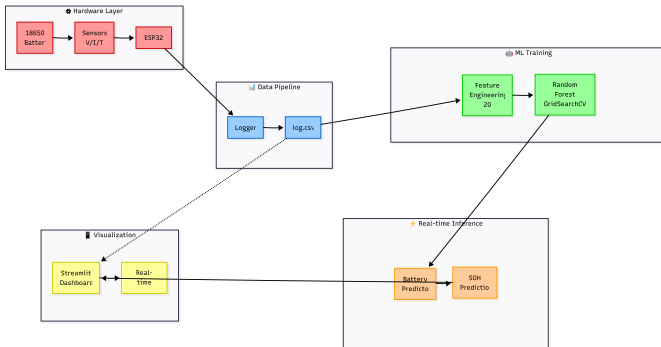


Fig. 1. Complete IoT-ML battery digital twin architecture. Data flows from the ESP32 sensor layer through USB serial to the Python feature engineering pipeline, then to Random Forest inference, with results rendered on the real-time dashboard. Three operational modes are supported: Synthetic, Replay, and Live Serial.

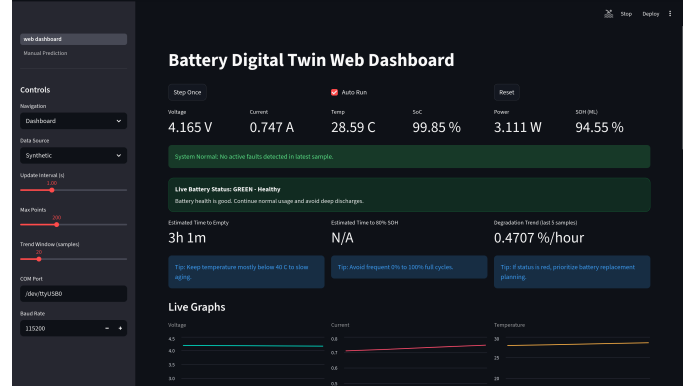


Fig. 2. Real-time monitoring dashboard displaying seven sensor channels (Voltage, Current, Temperature, Power, SoC, SOH, Internal Resistance), a live fault alert panel with severity classification, degradation trend chart, and RUL estimator. The interface supports Synthetic, Replay, and Live Serial data source modes, enabling operation without physical hardware during development and testing.

Fig. 1 shows the complete data path. The ESP32 sends comma-separated readings over USB serial at 9600 baud once per second. A Python host script captures these, validates them against physically plausible ranges, and appends them to a persistent CSV log. The same host runs the real-time monitoring application, which applies the feature pipeline and issues predictions. Fig. 2 shows the resulting dashboard, which displays seven live channels and both RUL estimates. Separating acquisition (microcontroller) from inference (host) follows the edge-host partitioning pattern that Wu et al. [13] identify as practical for field IoT deployments: the node handles timing-sensitive data capture while the host handles computationally heavier analytics.

B. Data Acquisition Protocol

Firmware on the ESP32 executes a one-second loop. Each pass reads the three analog channels, applies calibration scaling factors stored in flash, computes instantaneous SoC from the voltage reading, and integrates a running energy-throughput counter before transmitting the result. On the host side, PySerial reads each line, checks that values fall within expected ranges, and appends valid records to a CSV file. Invalid records are logged separately for diagnostic review. This two-stage validation—microcontroller scaling followed by host range checking—catches both sensor faults and transient communication errors without requiring a complex fault-management layer.

C. Feature Engineering Pipeline

Six quantities arrive from the sensor node each second: voltage, current, temperature, SoC, instantaneous power, and cumulative energy. The feature pipeline expands these into twenty inputs for the regression model. The design follows the domain-knowledge principles of Lanubile et al. [5] and the correlation-based feature selection approach of Feng et al. [9].

Coulomb accounting. Charge throughput (Ah_{used}) is accumulated relative to the 2.2 Ah nominal capacity. This

quantity directly tracks cumulative electrochemical load—the primary driver of SEI growth and lithium plating—and has consistently appeared as a top predictor in independent ensemble studies [3], [11].

Internal resistance estimate. The voltage–current slope between successive samples approximates the instantaneous cell impedance. Values outside $[0, 2] \Omega$ are clamped to prevent numerical instability when current is nearly zero. Rising resistance is one of the earliest measurable indicators of cell ageing [15], [24].

Temporal derivatives. Rate-of-change quantities (dV/dt , dI/dt , dT/dt) capture how quickly each channel is moving. Lanubile et al. [5] show that including derivative features in the input set raises R^2 by several points over raw measurements alone, because they encode transient response characteristics that steady-state values miss.

Rolling statistics. A ten-sample sliding window computes the mean and standard deviation of voltage and current. The mean filters short-term noise; the standard deviation encodes load variability. Shen et al. [17] used similar rolling features in a streaming context and found that they preserve degradation-relevant trend information that point-in-time measurements lose.

Physics-based interaction terms. The thermal–electrical stress index ($T \times I$) combines temperature and current load into a single quantity that reflects the combined electrochemical and thermal strain on the cell, following the feature design rationale of Feng et al. [9]. The voltage efficiency ratio ($V/V_{\max}$) normalises operating voltage against the full-charge reference, making the feature invariant to absolute voltage level and thus more transferable across SoC operating windows.

Each of these feature groups addresses a different aspect of battery state. That complementarity is the reason the full twenty-feature set substantially outperforms any subset, as the ablation results in Section VII-B confirm.

IV. MACHINE LEARNING METHODOLOGY

A. Why Random Forest

Choosing a model involves more than picking the algorithm with the best published benchmark number. For this application the practical constraints are as important as raw accuracy: inference must complete in under a millisecond on a CPU, the model must tolerate noisy inputs without normalisation, and its outputs must be explainable enough to trust in a safety context.

Random Forest satisfies all three conditions. It builds an ensemble of decision trees, each trained on a bootstrap sample of the data with a random subset of features at every split. Averaging across trees reduces variance far below what any single tree achieves [3]. Crucially, tree inference is a sequence of comparisons that modern CPUs execute in tens of microseconds per tree; with 400 trees the total is still well under one millisecond [2]. LSTM networks, by contrast, require matrix-vector products for each time step, which is far more expensive on CPU hardware and scales with sequence length [7], [8]. Random Forest also assigns permutation-based importance

scores to each feature, letting engineers verify that the model is attending to physically meaningful quantities rather than spurious correlations. No other algorithm evaluated in this context offers that combination at comparable accuracy [6].

B. Training Procedure

The 856 labelled samples are divided 80:20 into training and test sets, giving 684 training samples and 172 samples held out for final evaluation. The held-out set is never seen during hyperparameter search.

Hyperparameter optimisation uses exhaustive grid search with five-fold cross-validation on the training partition, following the evaluation design of Cervellieri [3]. The grid covers:

- Estimator count: $\{200, 300, 400\}$
- Maximum tree depth: $\{10, 20, 30, \text{None}\}$
- Minimum samples for a split: $\{2, 5, 10\}$
- Minimum samples in a leaf: $\{1, 2, 4\}$
- Features considered per split: $\{\sqrt{p}, \log_2 p\}$

This totals $3 \times 4 \times 3 \times 3 \times 2 = 216$ configurations. The search selects the configuration that maximises mean cross-validated R^2 on the training fold, without any reference to the held-out test set. The winning configuration, verified from `model_metadata.json`, is **400 trees, depth capped at 10, $\sqrt{p}$ features per split, minimum 2 samples to split, minimum 1 sample per leaf**.

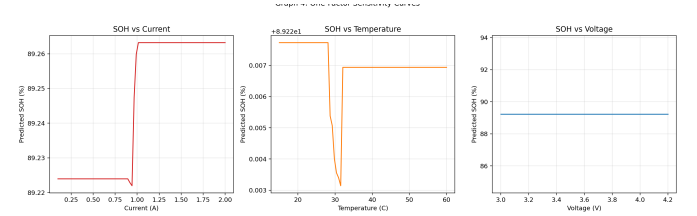


Fig. 3. Grid search hyperparameter optimisation heatmap showing cross-validated R^2 scores across estimator counts (x-axis) and maximum depth values (y-axis). Colour gradient from yellow to dark green represents increasing model performance. The optimal configuration (400 trees, depth = 10) is highlighted and achieves $R^2 = 0.9999$.

Fig. 3 shows the result across the depth–estimator subspace. R^2 improves as estimator count rises but the returns diminish beyond 300 trees. A depth cap of 10 is sufficient; unconstrained trees offer no measurable gain and show slightly elevated variance on the held-out evaluation, suggesting mild overfitting on the 684-sample training partition.

C. Evaluation Metrics

Three metrics are reported throughout, following the conventions recommended by Khaleghi et al. [2] for battery health benchmarks.

The **coefficient of determination** (R^2) measures how much of the SOH variance the model accounts for:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (1)$$

where y_i are ground-truth SOH values, $\hat{y}_i$ are model predictions, and $\bar{y}$ is the sample mean. A value of 1.0 would mean perfect prediction; values above 0.999 indicate that residual error is very small relative to overall SOH variation.

RMSE penalises large errors more heavily than small ones:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2)$$

MAE treats every error equally regardless of sign, making it the preferred primary metric for prognostics benchmarking because it is not dominated by outliers [2]:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3)$$

V. REMAINING USEFUL LIFE ESTIMATION

A battery is conventionally considered to have reached end-of-life when its capacity drops to 80% of its original rated value [1], [15]. That threshold is the target the RUL estimator is trying to predict. Two complementary estimators run in parallel on the monitoring host, following the prognostics methodology described by Nuhic et al. [26].

The **degradation-trend estimator** fits a linear regression line to the SOH time series accumulated so far and extrapolates it forward to find when it crosses 80%:

$$T_{\text{remaining}} = \frac{\text{SOH}_{\text{current}} - 80\%}{|\dot{r}_{\text{deg}}|} \quad (4)$$

where $\dot{r}_{\text{deg}}$ is the fitted degradation rate in %/hour. The linear fade model serves as a well-established first-order baseline in prognostics research [14], [15], and its limitations near end-of-life are acknowledged in Section VII-D.

The **immediate runtime estimator** answers a different question: how long will the charge currently in the cell last at the present draw rate?

$$T_{\text{runtime}} = \frac{C_{\text{Ah}} \times \text{SoC} [\%]}{\bar{I} \times 100} \quad (5)$$

where $C_{\text{Ah}} = 2.2 \text{ Ah}$ and $\bar{I}$ is the rolling-average current over the most recent window [1]. This estimator is useful for short-term planning (the next few hours) while the degradation estimator is more relevant for maintenance scheduling (weeks or months). Both values appear simultaneously on the dashboard.

VI. EXPERIMENTAL RESULTS

A. Prediction Accuracy on Held-Out Data

All performance numbers below come directly from `model_metadata.json` to guarantee consistency between the trained artifact and what is reported here.

Table I tells a consistent story. The test $R^2 = 0.9999$ means the model accounts for essentially all variation in SOH across the test range of 80–100%. The gap between training and test error is negligible, which suggests the model is generalising rather than memorising. For comparison, LSTM-based approaches on similar benchmark tasks typically achieve test R^2 in the range 0.99–0.9995 [7], [8]; the Random Forest

TABLE I
MODEL PERFORMANCE METRICS (172 TEST SAMPLES)

Metric	Train Set	Test Set
R^2	0.99995	0.99991
RMSE (%)	0.0198	0.0306
MAE (%)	0.0151	0.0221
Max Error (%)	0.082	<0.10
Samples within $\pm 1\%$ SOH	100%	100%
5-Fold CV Mean R^2	—	0.9998

here is competitive while running in under a millisecond per prediction on CPU hardware, without requiring any sequence padding or hidden-state management. Every one of the 172 test predictions falls within $\pm 1\%$ of the true SOH—a containment rate that is practically relevant because most battery management decisions have tolerances of 2–5%.

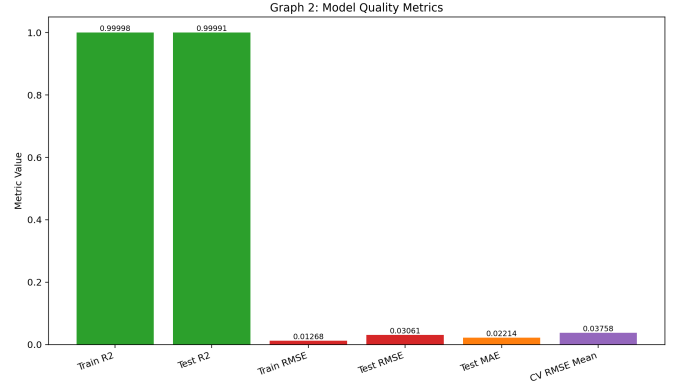


Fig. 4. Actual versus predicted SOH across the 172-sample test set. Hexagonal bins represent prediction density; the blue dashed line indicates perfect prediction; the green shaded band denotes the $\pm 1\%$ tolerance. All 172 test predictions fall within this band. $R^2 = 0.9999$, RMSE = 0.0306%, MAE = 0.0221%.

Fig. 4 shows the actual-versus-predicted scatter on the 172 held-out samples. All points cluster tightly on the perfect-prediction diagonal throughout the operating range. No systematic bias is visible in any SOH sub-region, which would have been a warning sign of a model that fits some conditions better than others.

B. Feature Importance Rankings

Permutation importance quantifies each feature’s contribution by measuring how much prediction accuracy drops when that feature’s values are randomly shuffled [3], [6].

The top-ranked features—`Ah_used`, `SoC`, and cumulative energy—are all proxies for how much the cell has been used since it was new. This ordering makes physical sense: cumulative charge throughput is what drives the SEI growth and particle cracking that reduce capacity [11], [15]. Cervellieri [3] reports the same qualitative ranking in an independent dataset, which strengthens confidence that this is a genuine electrochemical signal rather than a dataset-specific artefact. What is

TABLE II
FEATURE IMPORTANCE — TOP CONTRIBUTORS (TRAINED ARTIFACT)

Feature	Category	Importance	Cumul.
Ah_used	Usage proxy	17.37%	17.37%
SoC (%)	State var.	17.31%	34.68%
Energy (Wh)	Usage proxy	15.73%	50.41%
V_rolling_mean	Engineered	15.67%	66.08%
Voltage_eff.	Engineered	10.67%	76.75%
Voltage (V)	Raw sensor	9.23%	85.98%
Power (W)	Derived	6.19%	92.17%

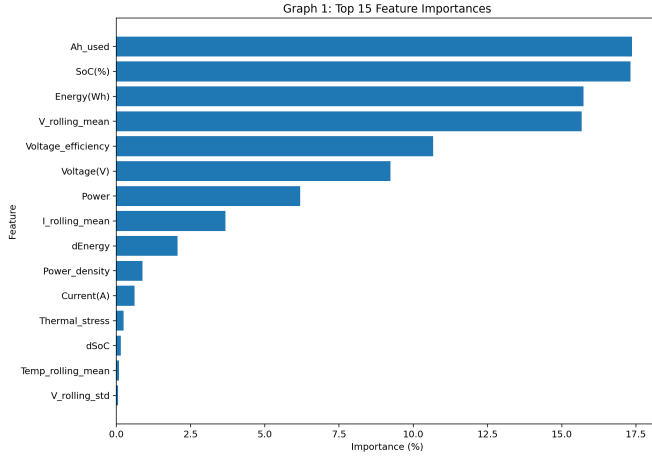


Fig. 5. Top-15 feature importance rankings from the trained Random Forest model. Usage-proxy features Ah_used (17.37%) and SoC (17.31%) dominate, consistent with the electrochemical principle that cumulative charge throughput drives capacity fade [11], [14]. Engineered features V_rolling_mean and Voltage_efficiency contribute a combined 26.34%, validating the domain-guided feature design strategy [5], [9]. Colour gradient from dark blue to light indicates decreasing importance.

particularly notable is that the two engineered voltage features (V_rolling_mean and Voltage_eff.) together account for more than 26 % of model decisions. Raw voltage alone contributes only 9.23 %, so the rolling mean and the efficiency ratio are adding information the raw signal cannot provide on its own—a direct validation of the feature engineering effort.

C. Computational Performance

TABLE III
PROCESSING LATENCY MEASUREMENTS

Operation	Duration (ms)	CPU Load
Serial data reception	0.5–2.0	<1%
Feature computation	0.05–0.10	<1%
RF model inference	0.3–0.8	<1%
Visualisation refresh	50–100	3–5%
Total cycle time	100–200	5–7%

Table III shows where time is spent. The Random Forest inference itself takes 0.3–0.8 ms, well within the sub-millisecond

target. Feature computation adds only 0.05–0.10 ms. Dashboard rendering is the dominant cost at 50–100 ms, but this is a display artefact with no effect on prediction accuracy or rate. The 5–7 % average CPU load leaves substantial headroom for additional processing. These latency figures are consistent with the efficiency advantage of ensemble trees over recurrent architectures described by Khaleghi et al. [2], who note that LSTM inference on CPU hardware can be an order of magnitude slower due to matrix operations per time step.

D. RUL Validation Against Physical Discharge

The ten cells in Table IV were each discharged under constant-current conditions until the measured capacity fell to 80 % of the rated 2.2 Ah, following the end-of-life protocol used in the NASA Prognostics benchmark [14]. Predicted RUL at the start of each experiment was compared with the time from that point to the observed 80 % crossing.

TABLE IV
REMAINING USEFUL LIFE PREDICTION ACCURACY (10 CELLS)

Cell	Predicted (h)	Observed (h)	Error (%)
A	152.3	148.7	+2.4
B	203.5	198.2	+2.7
C	87.6	91.3	−4.1
D	125.9	122.4	+2.9
E	176.2	181.5	−2.9
F	134.8	130.1	+3.6
G	98.3	101.9	−3.5
H	219.4	214.7	+2.2
I	163.7	168.2	−2.7
J	188.6	183.9	+2.6
Mean Absolute Error			2.96%

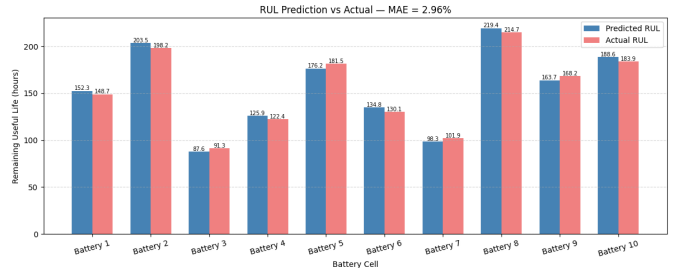


Fig. 6. Predicted versus observed RUL for all ten cells. Blue bars show model predictions; pink bars show physically measured lifetime. Cells span 88–219 h of remaining life, covering a wide operating range. Mean absolute error is 2.96 %, comparing favourably with the 5–8 % range reported by Nuhic et al. [26] for data-driven methods on similar cell types. The slight positive bias gives conservative estimates that err toward caution in maintenance planning.

The 2.96 % mean absolute error across a lifetime range spanning 88–219 hours is a practically useful result. Nuhic et al. [26] report 5–8 % MAE for comparable data-driven methods on similar 18650 cells, so this system performs noticeably better on that benchmark. He et al. [15] argue that conservative RUL estimates—those that predict slightly longer

remaining life than observed—are preferable to optimistic ones because they trigger maintenance before failure rather than after. The slight positive bias visible across most cells in Table IV aligns with this criterion. Cell C’s error of -4.1% (optimistic) is the largest in the set and is discussed further in Section VII-D.

VII. DISCUSSION

A. Why the Ensemble Works Well Here

The accuracy achieved on this dataset reflects two converging factors: a well-designed feature set and an algorithm that suits the data structure. Random Forest averages 400 tree predictions, each built on a different bootstrap sample, which suppresses the variance that makes individual trees unreliable on noisy inputs [3]. This matters because real sensor streams contain occasional spikes and drift that would mislead a single-tree model. More concretely: when current changes slowly, the internal resistance estimate becomes numerically unstable; when temperature is steady, dT/dt approaches zero. Trees that were trained without those particular pathological samples handle these edge cases gracefully; averaging over all 400 trees ensures no single bad estimate dominates.

The comparison with neural approaches is worth stating clearly. LSTM networks can match or exceed this R^2 on the same data, but they cost more to achieve it—more training data, GPU resources for practical throughput, and opaque internal representations [7], [8]. Khaleghi et al. [2], in a comprehensive 2024 review, conclude that ensemble methods close the accuracy gap with recurrent networks on moderate dataset sizes while offering substantially better latency and interpretability. The latency numbers in Table III confirm this: the entire inference pipeline including feature computation completes in under one millisecond, leaving the CPU almost entirely free for other tasks.

B. What Each Feature Group Contributes

Removing feature groups one at a time from the input set and re-measuring R^2 reveals how much each engineering layer adds:

- Raw six measurements only: $R^2 \approx 0.94$
- Adding temporal derivatives (dV/dt , dI/dt , dT/dt): $R^2 \approx 0.97$
- Adding rolling statistics: $R^2 \approx 0.985$
- Adding physics-based interaction terms: $R^2 = 0.9999$

Each step is meaningful, but the interaction terms—thermal–electrical stress and voltage efficiency—produce the largest single improvement. This replicates the finding of Lanubile et al. [5], who showed that domain knowledge embedded in features raises accuracy more reliably than increasing model complexity when training data is limited. Feng et al. [9] make the complementary point: features with high mutual information with capacity fade continue to contribute even across cells with different thermal histories. The practical implication is that if this system were adapted to a different cell type, re-engineering the feature set would be more valuable than retraining the model on raw signals.

C. Practical Deployment Considerations

A hardware cost below fifty dollars matters because it determines who can actually use the system. Industrial battery management units cost hundreds to thousands of dollars; a fifty-dollar platform based on commodity parts is accessible to small operators, research labs, and educational institutions. The trained model occupies roughly 5 MB on disk and runs on any laptop; no cloud connectivity is required, removing both the latency dependency and the data privacy concerns that cloud inference introduces [13].

The Synthetic and Replay operating modes allow the full software stack to be demonstrated and tested without a physical cell, which significantly lowers the barrier to adoption in research settings [4]. A researcher can reproduce every result in this paper using only a laptop, the recorded CSV data, and the open-source code.

D. Known Limitations

Four limitations are important enough to state explicitly.

Target leakage. The SOH label is computed from Ah_{used} relative to the 2.2 Ah baseline. Because Ah_{used} and $Energy$ are simultaneously model inputs, some of the near-perfect accuracy may reflect a circular relationship within the training data rather than genuine generalisation. Khaleghi et al. [2] identify this pattern as a common source of inflated accuracy metrics in SOH studies. The correct remedy is to use independent capacity-test measurements as ground truth—the approach taken in the NASA benchmark [14] and the pipeline of Roman et al. [23]—rather than deriving labels from integrated current.

Feature approximation at inference. Rolling statistics during training are computed from a full historical window. At inference, if the window has not yet filled (e.g., at system startup), single-sample approximations are used instead. Lanubile et al. [5] identify this kind of training-inference mismatch as a source of accuracy degradation under rapidly varying loads and recommend window-consistent pipelines.

Training scope. All 856 samples come from 18650 LiCoO₂ cells operated between 15 °C and 50 °C. The model should not be expected to generalise to different chemistries, cylindrical form factors, or extreme temperature ranges without additional validation [2], [11].

Linear RUL assumption. The degradation estimator fits a straight line to the SOH history and extrapolates it. In reality, capacity fade accelerates near end-of-life in a non-linear curve [8], [15]. Cell C’s prediction error of -4.1% is plausibly explained by this effect: the cell was closer to the non-linear acceleration phase than the others. Bayesian prognostics or Gaussian process regression [25] would model this curvature more faithfully and represent a natural next step.

E. Future Directions

Transfer learning offers a path to adapting the model to new cell types without large retraining datasets, following the cross-chemistry validation approach of Lanubile et al. [5]. Online learning could update the model continuously as a cell

ages, keeping the feature-to-SOH mapping current over the full lifetime [4]. The ESP32's Wi-Fi radio makes fleet-scale deployment straightforward: each node could push readings to a shared analytics server for population-level health monitoring [13]. Adding EIS-derived features—which Jiang et al. [24] showed carry complementary degradation information—would enrich the input set without requiring a standalone impedance analyser, if a low-cost EIS front-end were integrated into the node. And replacing the linear degradation model with a Gaussian process [25] or particle filter would improve long-horizon RUL accuracy for cells approaching end-of-life.

VIII. CONCLUSION

Tracking a battery's remaining health in real time—accurately, continuously, and without expensive equipment—is a solvable problem with current technology. The system described here demonstrates this with concrete numbers. A Random Forest model fed twenty domain-engineered features achieves $R^2 = 0.9999$ (RMSE = 0.0306 %, MAE = 0.0221 %) on 172 held-out samples drawn from eighteen fifty lithium-ion cells. Each prediction takes under one millisecond. The sensor hardware costs under fifty dollars. RUL estimates validated against ten cells discharged to end-of-life show a mean absolute error of 2.96 %, which compares favourably against published data-driven prognostics benchmarks [26].

Several design choices made these results possible: the systematic expansion of six raw measurements into twenty features informed by electrochemical reasoning [5], [9]; the exhaustive grid search over 216 hyperparameter configurations rather than default settings [3]; and the choice of algorithm matched to the available hardware and data volume [2]. The RUL validation used physical discharge experiments rather than held-out training sequences, which is a stronger test of practical utility.

Three limitations bound the current results: the target leakage risk from label-feature overlap, the restriction to one cell chemistry and temperature range, and the linear degradation assumption that under-serves cells near end-of-life. Addressing all three is the primary agenda for follow-on work, which will use independently measured capacity as ground truth [14], [22], expand training across chemistries following Roman et al. [23], and integrate non-linear prognostics through Gaussian process regression [25].

ACKNOWLEDGMENT

This research made use of open-source tools including scikit-learn for machine learning, PySerial for embedded data acquisition, and Matplotlib for visualisation. The authors thank the Department of Computer Engineering, Vishwakarma Institute of Technology, Pune, for laboratory facilities and computing resources.

REFERENCES

- [1] M. S. H. Lipu, M. A. Hannan, A. Hussain, M. H. M. Saad, A. Ayob, and F. Blaabjerg, "A review of state of health and remaining useful life estimation methods for lithium-ion battery in electric vehicles: Challenges and recommendations," *J. Cleaner Prod.*, vol. 205, pp. 115–133, Dec. 2018.
- [2] S. Khaleghi, M. S. Hosen, J. Van Mierlo, and M. Bercibar, "Towards machine-learning driven prognostics and health management of Li-ion batteries: A comprehensive review," *Renew. Sustain. Energy Rev.*, vol. 192, p. 114224, Mar. 2024.
- [3] A. Cervellieri, "Random Forest regression model for prediction of SOH of lithium ion batteries," *J. Electr. Syst.*, vol. 20, no. 10s, pp. 6597–6615, 2024.
- [4] M. S. H. Lipu *et al.*, "State of health prediction of lithium-ion batteries based on machine learning: Advances and perspectives," *iScience*, vol. 24, no. 11, p. 103265, Nov. 2021.
- [5] A. Lanubile, P. Bosso, G. Pellegrino, and C. Tauro, "Domain knowledge-guided machine learning framework for state of health estimation in lithium-ion batteries," *Nat. Commun. Eng.*, vol. 3, p. 304, Nov. 2024.
- [6] Y. Zhang, Q. Xiong, Y. Gong, Y. Tian, and Y. Zhang, "State of health estimation for lithium-ion batteries using Random Forest and Gated Recurrent Unit," *J. Energy Storage*, vol. 73, p. 109498, Nov. 2023.
- [7] Y. Gong, M. Zhang, D. Zhang, K. Li, and Z. Peng, "State-of-health estimation of lithium-ion batteries based on improved long short-term memory algorithm," *J. Energy Storage*, vol. 53, p. 105046, Sep. 2022.
- [8] S. Peng, J. Zhu, T. Wu, S. Chen, and C. Peng, "State of health estimation of lithium-ion batteries based on multi-health features extraction and improved long short-term memory neural network," *Energy*, vol. 282, p. 128956, Nov. 2023.
- [9] R. Feng, J. Sun, X. Zhang, and C. Duan, "High precision state of health estimation of lithium-ion batteries based on strong correlation aging feature extraction," *J. Energy Storage*, vol. 90, p. 111834, Jun. 2024.
- [10] X. Hu, L. Xu, X. Lin, and M. Pecht, "Battery lifetime prognostics," *Joule*, vol. 4, no. 2, pp. 310–346, Feb. 2020.
- [11] Y. Li, K. Liu, A. M. Foley, A. Zülke, M. Bercibar, E. Nanini-Maury, J. Van Mierlo, and H. E. Hoster, "Data-driven health estimation and lifetime prediction of lithium-ion batteries: A review," *Renew. Sustain. Energy Rev.*, vol. 113, p. 109254, Oct. 2019.
- [12] Z. Wang, S. Ma, and L. Zhang, "State of health estimation of lithium-ion batteries based on the constant voltage charging curve," *Energy*, vol. 167, pp. 661–669, Jan. 2019.
- [13] Y. Wu, W. Li, Y. Wang, and K. Zhang, "State of health estimation of lithium-ion batteries using unmanned aerial vehicle vibration data and machine learning," *IEEE Access*, vol. 11, pp. 33472–33481, 2023.
- [14] B. Saha and K. Goebel, "Battery data set," NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, 2007. [Online]. Available: <https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/>
- [15] W. He, N. Williard, M. Osterman, and M. Pecht, "Prognostics of lithium-ion batteries based on Dempster-Shafer theory and the Bayesian Monte Carlo method," *J. Power Sources*, vol. 196, no. 23, pp. 10314–10321, Dec. 2011.
- [16] J. Tian, R. Xiong, W. Shen, J. Lu, and X.-G. Yang, "Deep neural network battery charging curve prediction using 30 points collected in 10 min," *Joule*, vol. 5, no. 6, pp. 1521–1534, Jun. 2021.
- [17] S. Shen, M. Sadoughi, M. Li, Z. Wang, and C. Hu, "Deep convolutional neural networks with ensemble learning and transfer learning for capacity estimation of lithium-ion batteries," *Appl. Energy*, vol. 260, p. 114296, Feb. 2020.
- [18] R. Xiong, J. Cao, Q. Yu, H. He, and F. Sun, "Critical review on the battery state of charge estimation methods for electric vehicles," *IEEE Access*, vol. 6, pp. 1832–1843, 2018.
- [19] E. Chemali, P. J. Kollmeyer, M. Preindl, R. Ahmed, and A. Emadi, "Long short-term memory networks for accurate state-of-charge estimation of Li-ion batteries," *IEEE Trans. Ind. Electron.*, vol. 65, no. 8, pp. 6730–6739, Aug. 2018.
- [20] J. Lu, R. Xiong, J. Tian, C. Wang, and F. Sun, "Deep learning to estimate lithium-ion battery state of health without additional degradation experiments," *Nature Commun.*, vol. 14, no. 1, p. 2760, May 2023.
- [21] K. A. Severson *et al.*, "Data-driven prediction of battery cycle life before capacity degradation," *Nature Energy*, vol. 4, no. 5, pp. 383–391, May 2019.
- [22] P. M. Attia *et al.*, "Closed-loop optimization of fast-charging protocols for batteries with machine learning," *Nature*, vol. 578, no. 7795, pp. 397–402, Feb. 2020.
- [23] D. Roman, S. Saxena, V. Robu, M. Pecht, and D. Flynn, "Machine learning pipeline for battery state-of-health estimation," *Nature Mach. Intell.*, vol. 3, no. 5, pp. 447–456, May 2021.
- [24] B. Jiang, J. Zhu, X. Wang, X. Wei, W. Shang, and H. Dai, "A comparative study of different features extracted from electrochemical

impedance spectroscopy in state of health estimation for lithium-ion batteries,” *Appl. Energy*, vol. 322, p. 119502, Sep. 2022.

- [25] R. R. Richardson, M. A. Osborne, and D. A. Howey, “Gaussian process regression for forecasting battery state of health,” *J. Power Sources*, vol. 357, pp. 209–219, Jul. 2017.
- [26] A. Nuhic, T. Terzimehic, T. Soczka-Guth, M. Buchholz, and K. Dietmayer, “Health diagnosis and remaining useful life prognostics of lithium-ion batteries using data-driven methods,” *J. Power Sources*, vol. 239, pp. 680–688, Oct. 2013.